

**SIMATS ENGINEERING
ASSESSMENT TOOL 3**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO5: *Design AI-based systems using PySpark RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

Assessment Tool Weightage: 50%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Technical Presentation

Code of Conduct:

*I **T. Niranjan Kumar (192473007)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: T. Niranjan

SELECTED RESEARCH ARTICLE

Article: Zaharia, M. et al. Apache Spark: A Unified Engine for Big Data Processing. Communications of the ACM, 59(11), 56–65, 2016.

Review Focus: The paper is reviewed from the perspective of distributed AI systems, large-scale data processing, RDD-based computation and scalable intelligent applications.

DOI: 10.1145/2934664

Article Link: <https://doi.org/10.1145/2934664>

Note: This foundational peer-reviewed article is used to explain the technical concepts required for CO5. If the faculty assessment strictly requires an article published after a specific year, the citation should be replaced with an approved recent article while retaining the same review structure.

TASK 1 – ARTICLE SUMMARY

(a) Citation, Domain and Research Problem

The selected paper presents Apache Spark as a unified engine for large-scale data processing. Its domain is computing and big-data analytics distributed. The central problem addressed is that modern data-intensive applications need efficient, fault-tolerant and scalable processing for different workloads such as batch processing, SQL analytics, streaming and machine learning. The paper explains how a unified computational engine can simplify these workloads and improve performance.

(b) Article Summary

The main objective of the paper is to present Apache Spark as a unified platform for processing large volumes of data across a distributed cluster. Traditional systems often require separate engines for batch computation, interactive analytics, streaming and machine learning, which can increase development and operational complexity. Spark addresses this challenge by providing a common execution environment and high-level libraries while supporting fault tolerance and distributed execution. A major concept behind Spark is the Resilient Distributed Dataset (RDD), which represents a distributed collection of data that can be processed in parallel and reconstructed through lineage information when failures occur. The article also describes how Spark supports multiple workloads through components such as SQL processing, streaming and machine-learning libraries. The research is highly relevant to CO5 because scalable AI applications require efficient handling of large datasets, distributed computation and reliable data pipelines. Overall, the paper demonstrates how a unified distributed processing framework can provide the computational foundation needed by intelligent systems that analyse large-scale data and support real-world AI applications.

TASK 2 – TECHNICAL ANALYSIS

(a) Core Technical Concepts

1. Distributed Processing

Large datasets are divided into partitions and processed across multiple worker machines. This parallel approach reduces the amount of work performed by a single machine and allows the system to scale horizontally.

2. RDD Concept

An RDD is a distributed collection of data. RDDs support transformations that create new distributed datasets and actions that return results or trigger computation.

3. Important RDD Operations

- `map()`: Applies a function to each record.
- `filter()`: Selects records that satisfy a condition.
- `flatMap()`: Produces zero or more output elements from each input.
- `reduceByKey()`: Aggregates values associated with the same key.
- `join()`: Combines related key-value datasets.
- `collect()` and `take()`: Retrieve results from the distributed computation.

4. Fault Tolerance

RDD lineage records how an RDD was derived. If a partition is lost, the system can recompute the missing data from the transformations that created it.

5. Unified Processing

A unified engine allows related workloads to share infrastructure. This is useful for AI pipelines in which data preparation, feature engineering, model processing and analytics must operate at scale.

(b) Mapping to CO5 Concepts

- **PySpark**: Provides a Python interface for scalable Spark-based processing and makes distributed analytics accessible to AI applications written in Python.
- **RDD Operations**: Transformations and actions support parallel data preparation and aggregation.
- **Distributed AI Systems**: Spark can provide the large-scale data-processing layer used by AI models and intelligent applications.
- **Scalable Applications**: Cluster-based execution allows the application to process increasing data volumes by using additional computing resources.
- **Intelligent Agents**: Agent-based applications can use distributed data pipelines to obtain processed observations, aggregate environmental information and coordinate decisions.

(c) Technical Strength and Limitation

Strength: The unified architecture is a major strength because it reduces the need to manage separate systems for different data-processing workloads. The distributed model and lineage-based fault tolerance also support reliable large-scale computation.

Limitation: Spark provides a processing foundation but does not by itself solve every AI problem. Intelligent-agent coordination, domain-specific model design, data quality and safe decision-making require additional application-level components.

TASK 3 – RESULTS & CRITICAL EVALUATION

(a) Key Contributions

Contribution	Importance for CO5
Unified processing engine	Supports multiple data-intensive workloads using a common platform.
RDD abstraction	Enables distributed transformations, parallel processing and lineage-based recovery.
Fault tolerance	Improves reliability when processing data across multiple machines.
Scalability	Supports growth in data volume and computational requirements.
Higher-level libraries	Provides building blocks useful for analytics and machine-learning pipelines.

(b) Critical Evaluation

The paper is technically important because it explains a practical architecture for scalable data processing. Its design is especially useful when AI systems must process large datasets before or during model execution. However, distributed performance depends on cluster resources, partitioning strategy, data movement and workload characteristics. Therefore, a scalable framework does not automatically guarantee an optimal AI system.

For a complete evaluation of an AI application built on Spark, additional experiments should measure processing time, scalability with increasing nodes, memory use, fault recovery, model accuracy and end-to-end latency. If intelligent agents are involved, evaluation should also consider coordination delay, consistency of shared information and decision quality.

(c) Real-World Applicability

The concepts can be applied to recommendation systems, fraud detection, learning analytics, smart-city monitoring and disaster management. For example, large numbers of events can be distributed across worker nodes, transformed into useful features and aggregated before being supplied to AI models or decision agents.

Industry-Scale Challenges

- Data skew may cause some workers to receive much more work than others.
- Network communication and data shuffling can increase latency.
- Large clusters require monitoring, resource management and cost control.
- Real-time systems may require additional streaming and low-latency components.
- AI applications must also address privacy, security, bias and governance.

TASK 4 – REFLECTION & LEARNING OUTCOME

(a) Three Key Learning Insights

Insight 1 – RDD Operations: I understood that operations such as map, filter, join and reduceByKey are not only programming functions; they are building blocks for distributed data pipelines. They allow large datasets to be transformed and aggregated in parallel.

Insight 2 – Scalability: I learned that scalable AI requires more than selecting a good machine-learning algorithm. The data pipeline, cluster design, partitioning and fault tolerance are also important when the application operates on large volumes of information.

Insight 3 – Distributed AI Architecture: AI applications can use a layered architecture in which distributed processing prepares large-scale observations and intelligent agents or models make decisions using the processed information. This clearly connects data engineering with intelligent behaviour.

Additional Insight – Reliability: Lineage-based recovery shows why fault tolerance is important in distributed systems. A production AI application must continue processing even when individual machines fail.

(b) Proposed Extension

I would extend the reviewed architecture into a scalable multi-agent disaster-management system. PySpark would process large volumes of sensor, satellite and event data. RDD transformations would clean and aggregate observations by location and time. Specialised agents would then perform detection, prioritisation and communication tasks. A coordination component would combine agent outputs and produce response recommendations.

The extension is feasible because the distributed processing layer can scale with the number of observations, while specialised agents can divide the decision-making responsibilities. The expected impact is faster analysis of large data streams, better coordination between system components and improved support for time-sensitive response decisions.

FINAL CONCLUSION

This review connects the selected distributed computing research with the CO5 learning outcome. The main lessons are the importance of PySpark-style distributed processing, RDD transformations and actions, fault tolerance, scalability and their role as a foundation for intelligent applications. A complete distributed AI system combines these processing capabilities with suitable models or intelligent agents to convert large-scale data into useful decisions.